RAPORT DE PROIECT: APLICAȚIE PENTRU CASA INTELIGENTĂ (E HOME)

Student: Eivazov Iscander

GITHUB: https://github.com/iskander17Q/SmartHome-USV

- **Frontend: https://smarthome-usv-frontend.onrender.com/**
- **Backend: https://smarthome-usv.onrender.com/**

Pentru a accesa corect aplicația, vă rugăm să deschideți mai întâi ambele linkuri (frontend și backend). În caz contrar, acestea pot deveni inactive din cauza lipsei de utilizare (nu este implementat un mecanism de „ping" constant). După accesarea ambelor linkuri, vă rugăm să reîncărcați pagina.

1. Introducere și Obiectivele Proiectului

Proiectul „E Home" reprezintă o soluție software modernă pentru gestionarea dispozitivelor inteligente dintr-o locuință. Obiectivul principal al acestei lucrări a fost proiectarea, implementarea și punerea în producție (deployment) a unei aplicații web complete, având o arhitectură decuplată (client-server).

Sistemul permite utilizatorilor să se autentifice, să vizualizeze starea dispozitivelor din casă (cum ar fi lumini, televizoare, sisteme audio) și să le controleze de la distanță. Prin separarea clară a frontend-ului de backend, s-a urmărit obținerea unei aplicații scalabile, rapide, ușor de întreținut și de extins în viitor.

2. Arhitectura Sistemului și Tehnologii Utilizate

Aplicația a fost dezvoltată folosind un stack tehnologic modern, adaptat cerințelor actuale de pe piața dezvoltării web.

2.1. Backend (Server și Bază de Date)

Partea de server a fost realizată folosind limbajul **Python** și framework-ul **FastAPI**. Alegerea FastAPI se datorează performanței sale ridicate, suportului nativ pentru operațiuni asincrone și generării automate a documentației API (Swagger/OpenAPI).

Baza de date: Pentru stocarea structurată a datelor (utilizatori, dispozitive, stări, setări) s-a utilizat SQLite, o soluție ușoară, serverless și eficientă, ideală pentru mediul de dezvoltare și proiecte de dimensiuni medii.

Autentificare și Securitate: Securizarea accesului la API și protejarea datelor se face prin intermediul token-urilor JWT (JSON Web Tokens). Parolele utilizatorilor sunt criptate folosind algoritmi siguri (ex. bcrypt) înainte de a fi salvate în baza de date.

2.2. Frontend (Interfața cu Utilizatorul)

Interfața client a fost construită cu ajutorul framework-ului **Next.js** (bazat pe biblioteca **React**).

Design și UX: Aplicația beneficiază de un design modern, orientat către dispozitive mobile, folosind o temă întunecată (Dark Mode) elegantă și elemente vizuale de tip "glassmorphism" pentru a oferi o experiență utilizator (UX) premium și intuitivă.

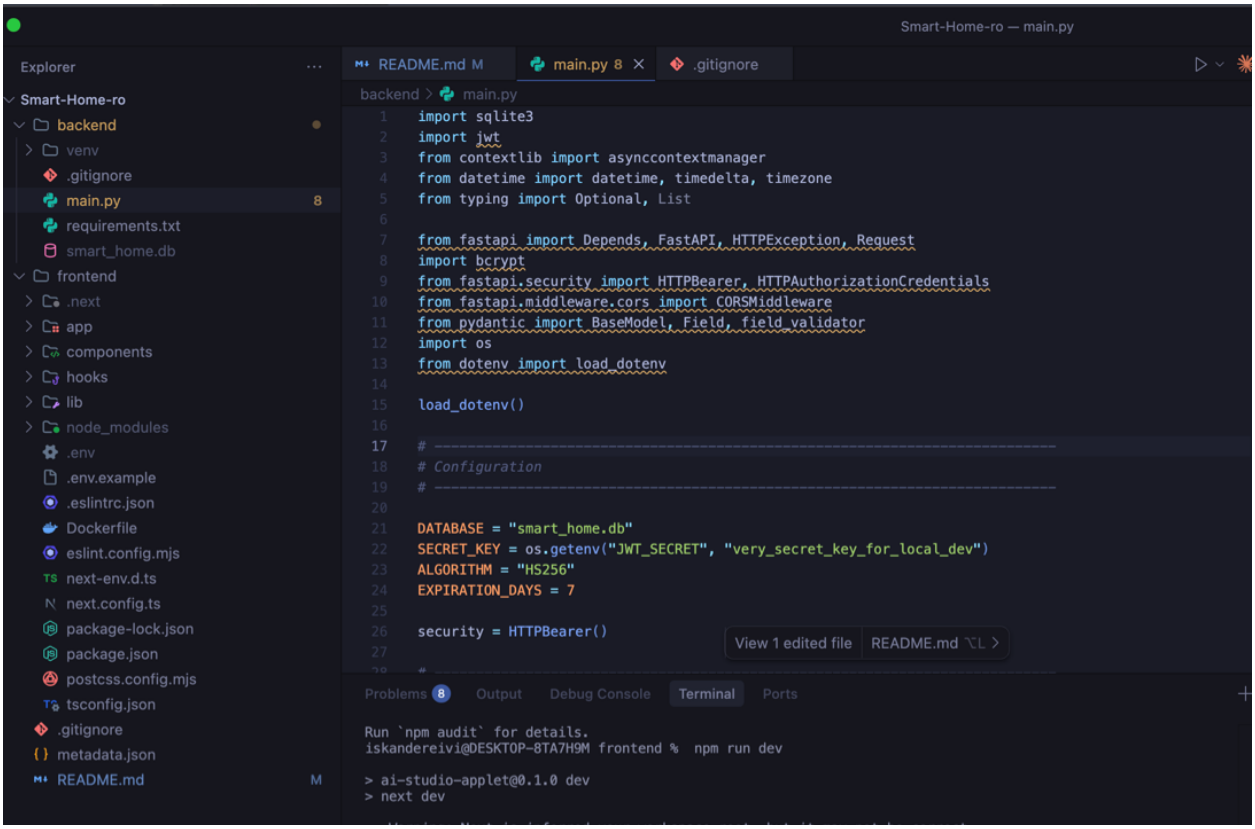
Comunicarea cu Serverul: Interogările către backend se realizează prin apeluri asincrone (fetch API) către endpoint-urile REST expuse de FastAPI. Acest mecanism permite actualizarea în timp real a stării dispozitivelor fără a fi necesară reîncărcarea paginii.

3. Procesul de Dezvoltare și Testare Locală

Pe parcursul dezvoltării, codul sursă a fost structurat în două directoare principale: `frontend` și `backend`. Dezvoltarea s-a realizat folosind editorul Visual Studio Code. Serverul de backend a rulat local pe portul 3001, folosind utilitarul `uvicorn` cu opțiunea de reîncărcare automată pentru a facilita testarea rapidă a modificărilor. În paralel, frontend-ul a rulat pe portul 3000.

Am implementat și testat cu succes funcționalități cheie precum:

- Înregistrarea și autentificarea sigură a utilizatorilor;
- Dashboard personalizat care afișează dispozitivele disponibile pe camere (ex. "Muzica", "Televizorul");
- Interacțiunea utilizatorului prin butoane de tip "toggle" pentru pornirea/oprirea rapidă a dispozitivelor.

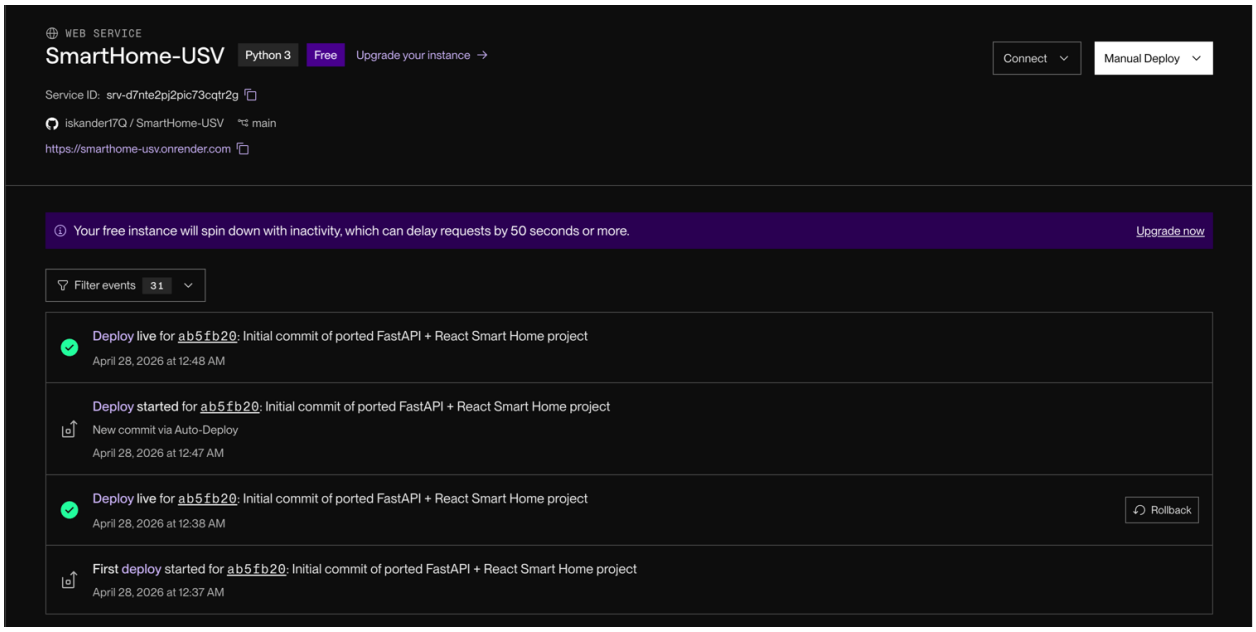


4. Implementarea în Mediul de Producție (Deployment pe Render.com)

Pentru a face aplicația accesibilă la nivel global, am ales platforma de cloud hosting **Render.com** pentru procesul de deployment. Această decizie ne-a permis configurarea unui flux de integrare continuă și implementare continuă (CI/CD) conectat direct la repository-ul proiectului de pe GitHub.

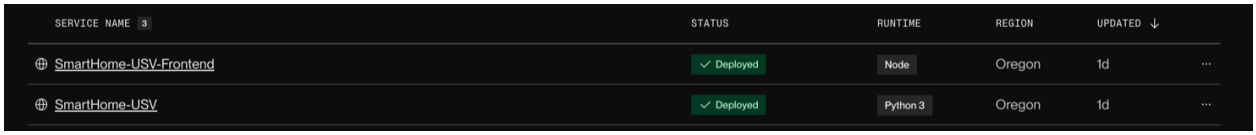
4.1. Configurarea și Deploy-ul Backend-ului (SmartHome-USV)

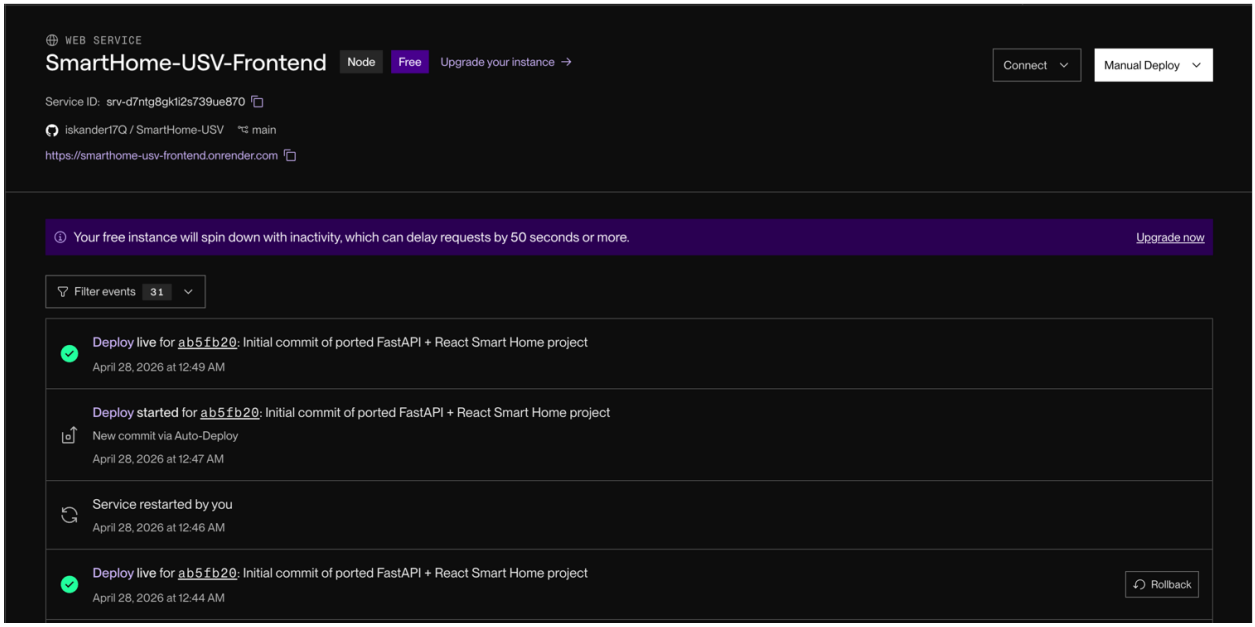
Backend-ul a fost configurat ca un Web Service separat pe Render, rulând în mediul Python 3. Procesul de build a fost configurat să instaleze automat toate dependențele listate în fișierul `requirements.txt`. Am adăugat variabilele de mediu necesare, inclusiv cheile secrete pentru generarea token-urilor JWT. Render monitorizează ramura principală (main) și inițiază automat procesul de deploy la fiecare commit nou (`Auto-Deploy`), asigurând astfel că serverul rulează mereu cea mai recentă versiune a codului.



4.2. Configurarea și Deploy-ul Frontend-ului (SmartHome-USV-Frontend)

În mod similar, aplicația client a fost publicată ca un Web Service distinct, folosind mediul Node.js. Procesul de build pe Render rulează comenzile standard `npm install` urmate de `npm run build` pentru a genera versiunea optimizată pentru producție (production bundle) a aplicației Next.js. Frontend-ul este configurat să comunice direct cu URL-ul public al backend-ului, setat prin intermediul variabilelor de mediu specifice producției.

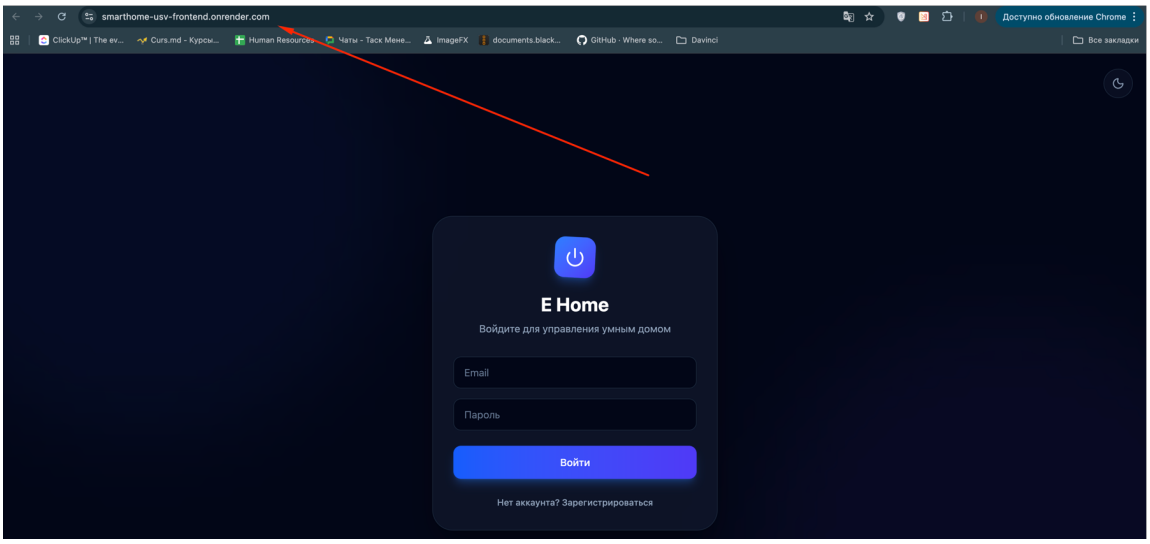


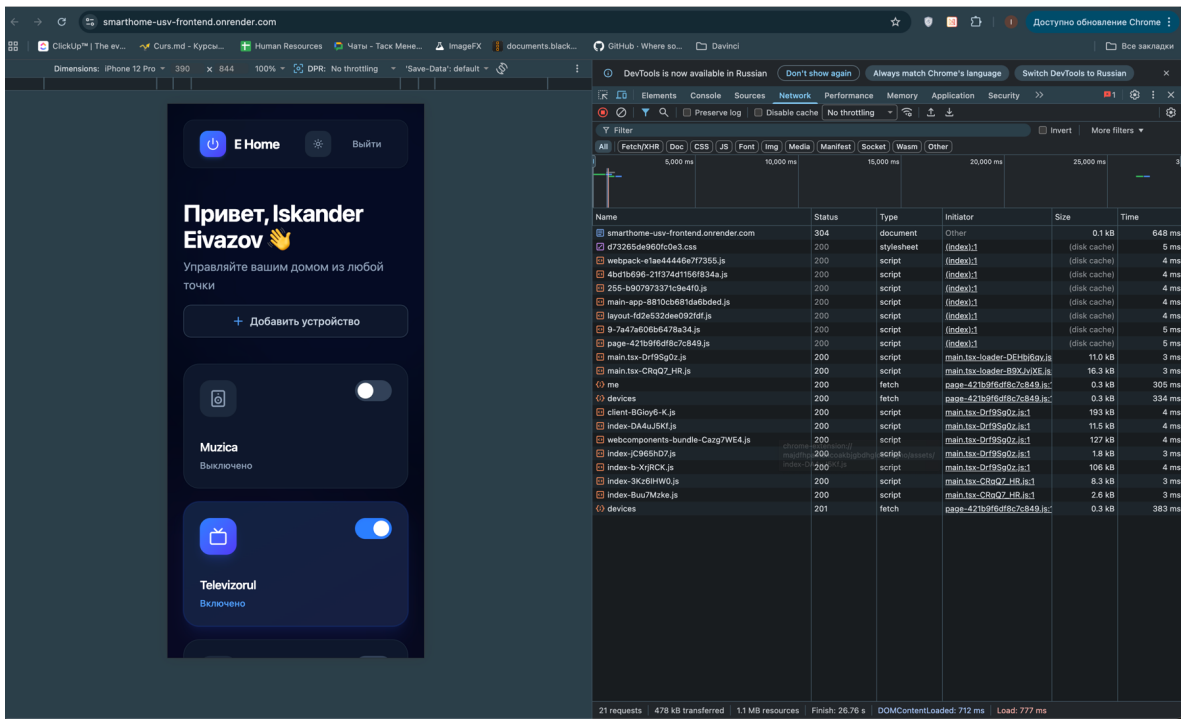


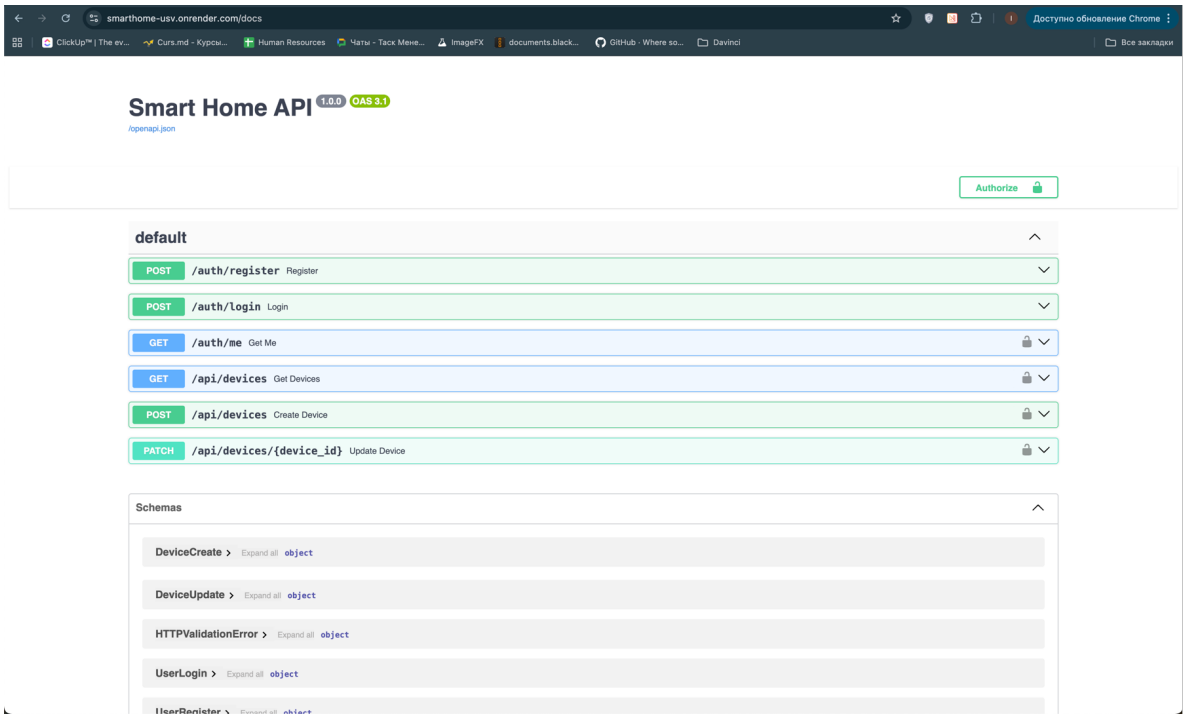
5. Testare, Performanță și Analiza Interacțiunii

După finalizarea cu succes a procesului de deployment, am efectuat teste practice pe mediul live (`smarthome-usv-frontend.onrender.com`). Folosind instrumentele pentru dezvoltatori integrate în browser (Chrome DevTools - tab-ul Network), am analizat și optimizat traficul de rețea.

S-a observat că la încărcarea inițială, aplicația livrează rapid resursele statice (fișiere CSS și scripturi JavaScript segmentate eficient de Next.js). Interacțiunile utilizatorului, cum ar fi obținerea detaliilor profilului (`/me`) sau a listei de dispozitive (`/devices`), generează cereri HTTP rapide care primesc răspunsuri cu codul de status 200 (OK) într-un timp de ordinul milisecundelor. Aceasta demonstrează o integrare reușită, o comunicare stabilă și o latență redusă între frontend și backend, chiar și în mediul cloud.

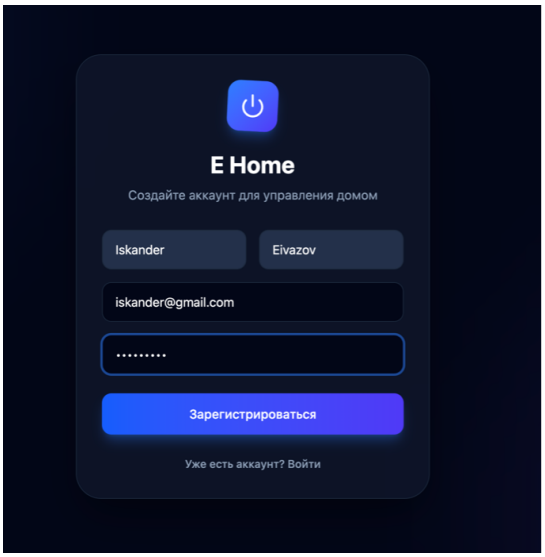
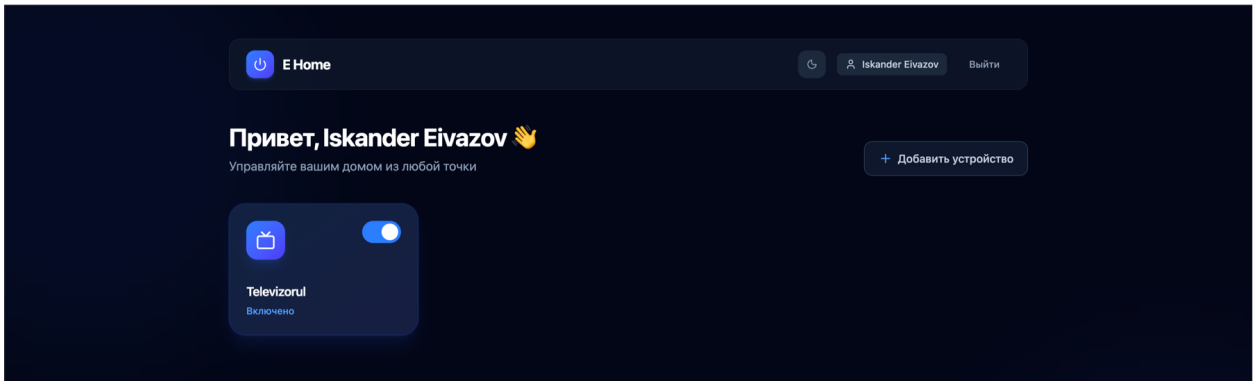
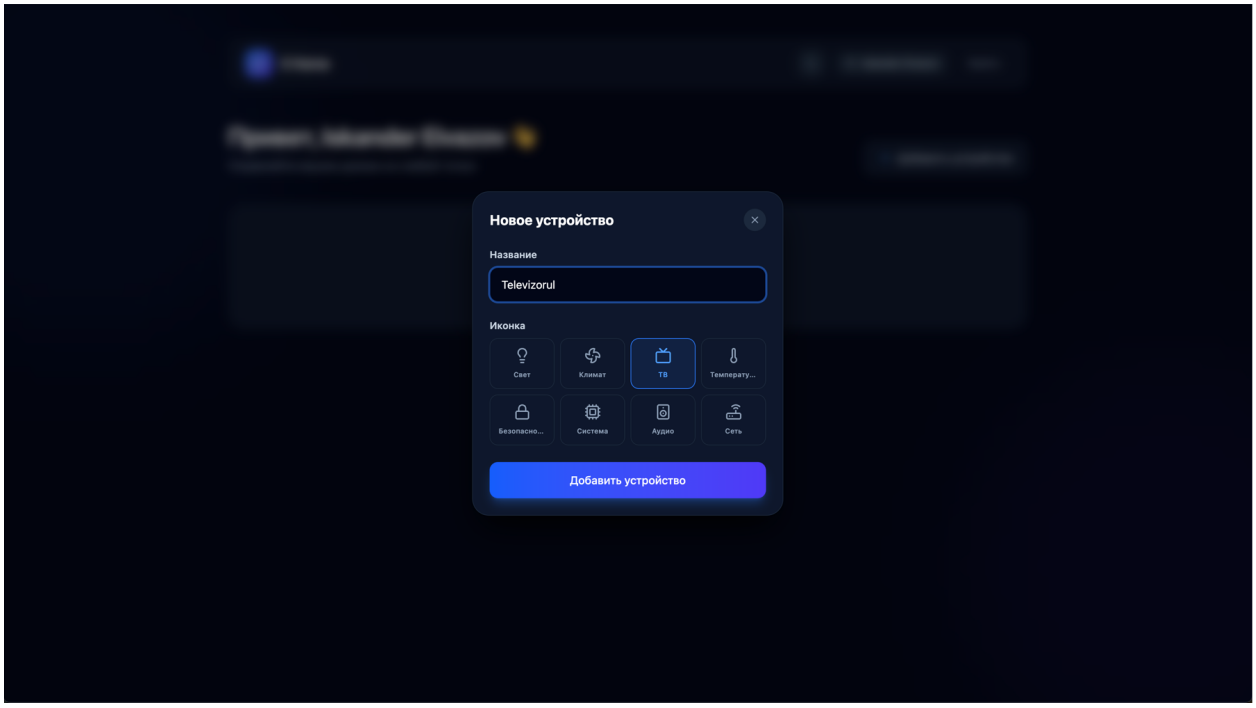






6. Concluzii și Direcții Viitoare

Proiectul "E Home" a fost realizat cu succes, acoperind întregul ciclu de viață al dezvoltării software, de la concept și implementare locală, până la arhitectura cloud și deployment.

Prin utilizarea FastAPI pentru performanța și structura solidă a backend-ului, împreună cu Next.js pentru interfața dinamică și reactivă, am creat o soluție robustă. Procesul de deployment pe Render.com a validat capacitatea sistemului de a rula în producție. În starea actuală, aplicația este complet funcțională și oferă o bază excelentă pentru extinderi viitoare. Printre direcțiile viitoare de dezvoltare se numără integrarea senzorilor fizici reali prin protocoale IoT (ex. MQTT),

adăugarea unor module de analiză avansată a consumului de energie și implementarea asistenților vocali.